

ÉVALUATION — NEXT.JS

Mini-projet Next.js

Application météo

Concevoir une application web complète permettant de rechercher une ville, de consulter ses conditions météorologiques et ses prévisions, et de gérer une liste de villes favorites — développée avec **Next.js**, **TypeScript** et l'**App Router**.

Recherche de villes

Géocodage et suggestions en temps réel

Météo & Prévisions

Conditions actuelles et prévisions journalières

Gestion des favoris

Persistance et accès rapide aux villes

Fonctionnalité originale

Une valeur ajoutée au choix de l'étudiant

Fonctionnalités attendues

L'application météo Next.js sera dotée d'une suite de fonctionnalités robustes, conçues pour offrir une expérience utilisateur fluide et informative. Ces fonctionnalités couvrent la recherche de villes, l'affichage détaillé de la météo et des prévisions, la gestion personnalisée des favoris, et une innovation distinctive propre à l'étudiant.

1

Page d'accueil

Point d'entrée de l'application, offrant un accès rapide aux informations clés et aux outils de navigation.

- **Recherche de villes** : Champ de recherche intuitif avec suggestions en temps réel (géocodage) et auto-complétion.
- **Affichage des favoris** : Section dédiée présentant les villes favorites avec accès instantané à leurs conditions météo.
- **Navigation** : Liens clairs vers la page de détail de chaque ville et autres sections pertinentes.

2

Page de détail

Vue approfondie des conditions météorologiques d'une ville spécifique.

- **Conditions actuelles** : Température, ressenti, humidité, pression, vent, indice UV et état du ciel.
- **Prévisions journalières** : Prévisions sur 5 à 7 jours avec températures min/max et icônes météo.
- **Lever/coucher du soleil** : Informations précises sur les phases solaires.
- **Carte ou localisation** : Option pour afficher la ville sur une carte basique.

3

Gestion des favoris

Permet aux utilisateurs de personnaliser leur expérience en sauvegardant les villes qui les intéressent.

- **Ajout / Suppression** : Fonctionnalité accessible depuis la page de détail ou la page d'accueil.
- **Persistence** : Sauvegarde via le stockage local du navigateur pour persister entre les sessions.
- **Indicateurs visuels** : Icône étoile ou cœur indiquant clairement le statut favori sur toutes les pages.

4

Fonctionnalité originale

Fonctionnalité unique démontrant créativité et capacité d'innovation, ajoutant une valeur distincte à l'application.

- **Exemples** : Historique des recherches, comparaison météo entre deux villes, alertes personnalisées, intégration d'une API tierce (qualité de l'air, pollen), ou mode "voyage" suggérant la meilleure période pour visiter une ville.
- **Valeur ajoutée** : Doit aller au-delà des attentes de base et enrichir significativement l'expérience utilisateur.

Contraintes techniques

Le succès du mini-projet Next.js repose sur l'adhésion à un ensemble de contraintes techniques rigoureuses. Ces lignes directrices garantissent non seulement la robustesse et la maintenabilité de l'application, mais aussi une expérience utilisateur optimale et une adhésion aux meilleures pratiques de développement moderne. De la sélection des technologies aux conventions de codage, chaque aspect est conçu pour favoriser une application performante, évolutive et facile à auditer.

Technologies Clés



Next.js
Framework React pour une application robuste



TypeScript
Typage statique pour une meilleure fiabilité du code



React
Bibliothèque pour la création d'interfaces utilisateur



App Router
Nouvelle architecture de routage et de rendu

Architecture et Composants

- **Composants réutilisables:** Développer des composants UI génériques et autonomes
- **Pas de duplication d'appels API:** Implémenter des mécanismes de cache

Exigences TypeScript

Définitions de types propres
Assurer la clarté et la cohérence des types à travers l'application.

Utilisation minimale de 'any'
Éviter le type 'any' pour maintenir la sécurité de type.

Fonctionnalités Next.js Spécifiques

1

Routes dynamiques
Gérer les URLs basées sur le contenu (par exemple, `/ville/[nom]`).

2

Server Components et Client Components
Optimiser le rendu en utilisant les composants appropriés pour chaque tâche.

3

États de chargement
Fournir un feedback visuel pendant le chargement des données (par exemple, `loading.tsx`).

4

Gestion des erreurs
Implémenter des pages d'erreur personnalisées (par exemple, `error.tsx`, `not-found.tsx`).

Interface Utilisateur (UI/UX)

Clarté et Cohérence
Conception intuitive avec une harmonie visuelle sur toutes les pages.

Réactivité (Responsiveness)
L'application doit s'adapter et être entièrement fonctionnelle sur divers appareils (mobiles, tablettes, ordinateurs de bureau).

Organisation du Projet

- ☐ Structure de projet propre
- ☐ Pas de code mort
- ☐ Pas de secrets dans le code

Livrables & README.md

Cette section détaille les attentes concernant la soumission de votre projet, incluant la structure du dépôt Git et les informations essentielles à inclure dans votre fichier README.md. Une documentation claire est cruciale pour la compréhension et l'évaluation de votre travail.

Soumission du Projet

La livraison de votre travail se fera exclusivement via le lien de votre dépôt Git public. Assurez-vous que le dépôt est accessible et contient tous les éléments nécessaires.

**Code source complet**
L'intégralité du code source de l'application.

**README.md détaillé**
Un fichier de documentation obligatoire et complet.

**Captures d'écran**
Des captures d'écran pertinentes de l'application.

**.gitignore configuré**
Un fichier .gitignore correctement mis en place.

Exigences du Fichier README.md

Votre fichier README.md est la carte d'identité de votre projet. Il doit être concis mais complet, guidant le lecteur à travers les aspects clés de votre application.

 Nom du Projet & Description Indiquez le nom de l'application et fournissez une brève description de son objectif et de ses fonctionnalités principales.	 Fonctionnalités Implémentées Listez toutes les fonctionnalités implémentées, y compris les éléments attendus et toute amélioration additionnelle.
 Fonctionnalité Originale Décrivez en détail la fonctionnalité originale intégrée, sa valeur ajoutée et son implémentation technique.	 Technologies Utilisées Énumérez les technologies, frameworks et bibliothèques principales : Next.js, TypeScript, App Router, APIs externes, etc.
 Instructions d'Installation & Lancement Étapes claires pour cloner le dépôt, installer les dépendances et lancer l'application localement.	 Variables d'Environnement Liste des variables requises et leur configuration (ex : fichier .env.local).
 Choix d'Architecture & Composants Décisions architecturales clés : utilisation et distinction entre Server Components et Client Components.	 Captures d'Écran Captures de haute qualité illustrant les interfaces clés : • Page d'accueil avec recherche et favoris • Résultats de recherche et détail d'une ville • Gestion de la liste des villes favorites • Fonctionnalité originale en action

 **Important :** N'incluez jamais de dossiers générés (node_modules, dossiers de build), de fichiers temporaires, ou de secrets/clés API dans votre dépôt Git. Utilisez .gitignore et les variables d'environnement.

Critères d'évaluation

L'évaluation de votre mini-projet d'application météo Next.js reposera sur une série de critères techniques et fonctionnels visant à garantir la qualité, la robustesse et l'efficacité de votre solution. Chaque aspect, de la conception du code à l'expérience utilisateur, sera pris en compte pour mesurer l'atteinte des objectifs pédagogiques et la démonstration de vos compétences.



Qualité du code et architecture

Clarté, maintenabilité, modularité et respect des principes de conception logicielle, y compris la séparation des préoccupations et l'organisation des composants.



Implémentation des fonctionnalités

Vérification de la présence et du bon fonctionnement de toutes les fonctionnalités requises : recherche de ville, affichage météo, prévisions, gestion des favoris et la fonctionnalité originale.



Meilleures pratiques Next.js

Utilisation appropriée de l'App Router, du rendu côté serveur (SSR) ou client (CSR) selon les besoins, de l'optimisation des images et des stratégies de chargement des données.



Utilisation de TypeScript

Qualité des définitions de types, prévention des erreurs à la compilation et utilisation efficace des fonctionnalités avancées de TypeScript pour une plus grande robustesse.



UI/UX et réactivité

Esthétique générale, ergonomie de l'interface utilisateur, fluidité de navigation et adaptation de l'application à différentes tailles d'écran (responsive design).



Organisation du projet et documentation

Clarté de la structure des dossiers, lisibilité du code, présence de commentaires pertinents et qualité de la documentation (notamment le fichier README.md).



Historique Git et messages de commit

Un historique Git propre et des messages de commit significatifs qui retracent l'évolution du projet de manière compréhensible.

Contexte & Ressources

Pour développer notre application météo interactive, nous nous appuyerons sur l'API **Open-Meteo**, une source fiable et gratuite pour les données météorologiques. Cette section détaille les ressources clés nécessaires pour la récupération des informations de localisation et de prévisions climatiques.

 La documentation complète de l'API Open-Meteo est disponible sur : <https://open-meteo.com/>

APIs utilisées

Deux APIs principales seront utilisées pour construire les fonctionnalités de notre application :



API de Géocodage

Cette API nous permettra de convertir les noms de villes entrés par l'utilisateur en **coordonnées géographiques** (latitude et longitude). Ces coordonnées sont essentielles pour la recherche et la récupération précise des données météorologiques.



API Météo

Une fois les coordonnées géographiques obtenues, cette API sera utilisée pour récupérer les **conditions météorologiques actuelles** ainsi que les prévisions journalières pour la ville sélectionnée, alimentant ainsi l'affichage principal de notre application.